

Milestone Report: RL-Based Selection of Domain-Specialized EAGLE3 Drafters

김재연 박준호 오윤상 이도규 이수정

1 Revised Project Direction

Our project studies adaptive drafter selection for speculative decoding. In speculative decoding, a small drafter proposes future tokens and a larger target model verifies them in parallel. The speedup depends on drafter-target alignment: if the proposed tokens are likely under the target model, more tokens are accepted per verification step.

The original proposal focused on reducing the number of active drafters with an active-set penalty. After further analysis, we found that active-set size alone is not a reliable proxy for actual serving performance. If only one drafter is executed at a time, the direct latency cost is determined more by the selected drafter and serving implementation than by the number of candidate drafters. Therefore, we revise the project goal: rather than optimizing the number of active drafters, we will keep a fixed pool of domain-specialized drafters and compare RL-style selection policies for choosing which drafter to use.

This direction is motivated by two recent adaptive speculative decoding systems. MetaSD [1] shows that specialized drafters can have strong domain-dependent performance and formulates drafter selection as a multi-armed bandit problem. ATLAS [2] similarly argues that static speculators can lose alignment as workloads shift, although it addresses the issue through a static/adaptive speculator pair with runtime learning. Instead of assuming the same checkpoints are available, we plan to construct our own controlled drafter pool using EAGLE3 [4] and evaluate whether domain specialization appears in our setup.

2 Planned Setup

We will use one fixed target verifier model X , such as a Llama-3.1-class 7B–8B instruction model compatible with SGLang/SpecForge. For this target, we will train multiple EAGLE3 drafters from the same base architecture but on different domain-specific datasets:

- Y_{gsm8k} : trained on GSM8K-style math reasoning data.
- Y_{mmlu} : trained on MMLU-style broad knowledge and exam QA data.
- Y_{sharegpt} : trained on ShareGPT-style general instruction/chat data.
- $Y_{\text{code/trans}}$: trained on either code or translation data, to be finalized based on dataset and training stability.
- Y_{all} : trained on the union of all domain datasets.

We removed the previously considered random-prompt domain because it is not a coherent specialization target. The fourth domain will be chosen to be distributionally distinct from GSM8K, MMLU, and ShareGPT; code generation or translation are the current candidates.

To make the comparison fair, all five drafters will use comparable training budgets. Ideally, we will match the number of optimizer steps or the approximate number of training tokens. Since domain-specific datasets are smaller than the all-domain union, a domain-specific drafter may revisit its 500-example dataset for more epochs, while Y_{all} trains for fewer epochs over the 2000-example

union. For example, a domain-specific drafter might train for 16 epochs over 500 examples, while Y_{all} trains for 4 epochs over 2000 examples, giving the same rough number of example exposures.

3 Progress Since Proposal

The main progress since the proposal is a more concrete and implementable experimental design. We reviewed the MetaSD and EAGLE/EAGLE3 setting and concluded that the project should first build a controlled heterogeneous drafter pool, then evaluate selection policies over that pool. We have fixed the planned evaluation structure. Each domain will have a small training subset and a held-out evaluation subset. The held-out split is important because evaluating on the same examples used for drafter fine-tuning would confound specialization with memorization. The milestone-stage target sizes are:

- 500 training examples per domain.
- 100–200 held-out evaluation examples per domain.
- 2000 total training examples for the all-domain drafter.

We also prepared the training and logging plan. EAGLE3 is a white-box speculative decoding method, so the drafter is trained using features from the fixed target model. The expected pipeline is: prepare domain data, optionally regenerate responses using the target model, extract or stream target hidden states/logits through SpecForge, train each EAGLE3 drafter, and then evaluate each drafter with the same target model and decoding settings. The actual five training runs have not yet been completed and remain the next implementation step.

4 Evaluation Plan

The first evaluation artifact will be a domain-by-drafter matrix. Each row is an evaluation domain and each column is a trained EAGLE3 drafter. The primary metric is accepted tokens per verification step:

$$\text{ATVS} = \frac{\text{number of accepted draft tokens}}{\text{number of target verification steps}}. \quad (1)$$

We will also record acceptance rate, relative speedup over autoregressive decoding, and latency statistics if the serving stack exposes stable timing measurements.

Eval domain	Y_{gsm8k}	Y_{mmlu}	Y_{sharegpt}	$Y_{\text{code/trans}}$	Y_{all}
GSM8K	TBD	TBD	TBD	TBD	TBD
MMLU	TBD	TBD	TBD	TBD	TBD
ShareGPT	TBD	TBD	TBD	TBD	TBD
Code/translation	TBD	TBD	TBD	TBD	TBD

Table 1: Planned domain-by-drafter matrix. Each entry will report accepted tokens per verification step.

The expected result is that a domain-specialized drafter can outperform Y_{all} on at least some matching domains, while Y_{all} may be more robust on average. If this pattern appears, it creates room for adaptive selection. If it does not appear, that negative result is still informative: it would suggest that all-domain EAGLE3 training is sufficient in our setting.

5 RL Selection Plan

After training and evaluating the five drafters, we will use the fixed drafter pool for online selection. The selection granularity is still under investigation. Per-request selection is easier to implement

because the router chooses one drafter for the whole response. Per-step selection is closer to MetaSD because the policy can switch drafters at each speculative decoding round, but it may require deeper changes to the serving loop. We will decide between these options after checking the runtime integration cost.

The candidate RL and bandit methods are also tentative. We currently plan to start with simple baselines and add more methods only if the implementation remains stable:

- Uniform random selection.
- Greedy selection from observed rewards.
- UCB.
- Thompson sampling or EXP3 as optional alternatives.

The reward will initially be accepted tokens per verification step or an equivalent acceptance-based signal. If per-step selection is feasible, we may also consider denser alignment feedback similar to MetaSD’s block divergence reward.

6 Next Steps

The remaining work is:

1. Finalize the target model X and the fourth domain.
2. Build the domain-specific datasets and held-out evaluation splits.
3. Complete the SpecForge/EAGLE3 training environment.
4. Train Y_{gsm8k} , Y_{mmlu} , Y_{sharegpt} , $Y_{\text{code/trans}}$, and Y_{all} with matched training budgets.
5. Produce the domain-by-drafter matrix.
6. Decide whether dynamic selection will be implemented per request or per speculative decoding step.
7. Implement and compare a small set of RL/bandit selection policies.

References

- [1] T. Kim, H. Jung, and S.-Y. Yun. Multi-Drafter Speculative Decoding with Alignment Feedback. arXiv:2604.05417, 2026.
- [2] J. Wang et al. AdapTive-LeArning Speculator System (ATLAS): A New Paradigm in LLM Inference via Runtime-Learning Accelerators. Together AI Blog, 2025. <https://www.together.ai/blog/adaptive-learning-speculator-system-atlas>.
- [3] Y. Leviathan, M. Kalman, and Y. Matias. Fast Inference from Transformers via Speculative Decoding. ICML, 2023.
- [4] Y. Li et al. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. arXiv:2503.01840, 2025.
- [5] D. Hendrycks et al. Measuring Massive Multitask Language Understanding. ICLR, 2021.
- [6] K. Cobbe et al. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021.